

# Air Gesture Ammu

Project description, working diagrams, requirements, configuration, build instructions, and operating guide

| VOICE                        | AIR GESTURES                  | SENSORS                    | LOST PHONE               |
|------------------------------|-------------------------------|----------------------------|--------------------------|
| Commands and spoken feedback | Swipe, palm, and fist control | Tilt, shake, and proximity | Scheduled location email |

Prepared from the implemented Android source code

Repository: [github.com/munagapati-benarjibabu/Air-gesture-ammu](https://github.com/munagapati-benarjibabu/Air-gesture-ammu)

# 1. Project Description

Air Gesture Ammu is an Android demonstration application for touchless phone interaction. It combines Android speech services, CameraX, MediaPipe gesture recognition, device sensors, accessibility actions, call controls, and a foreground lost-phone location service in one Jetpack Compose application.

## Objectives

The project explores practical accessibility and hands-free workflows using hardware already present on a standard smartphone. The design keeps recognition and action execution separated so gestures, voice commands, and sensor events can share the same Android action layer.

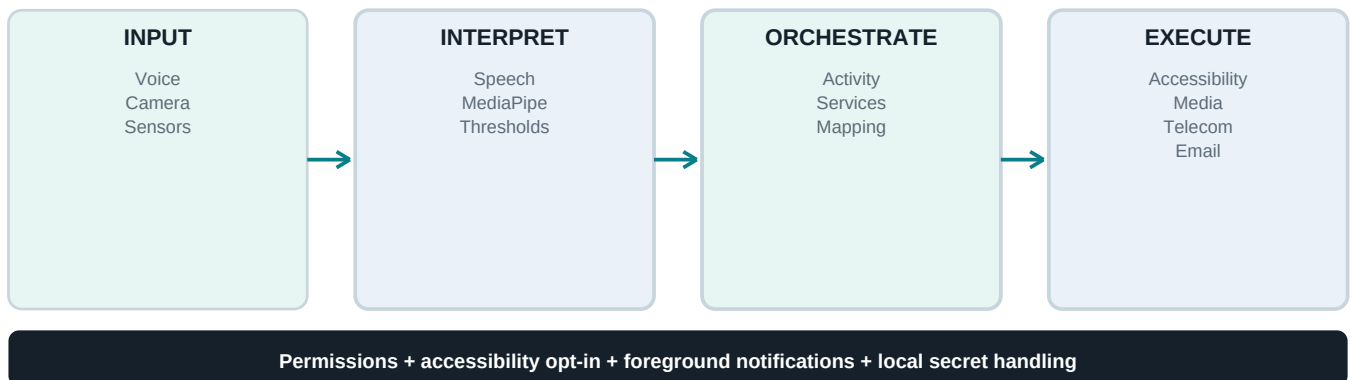
- Control scrolling and navigation without touching the display.
- Launch common applications and control media using voice or gestures.
- Use accelerometer and proximity events as alternative input channels.
- Send scheduled location emails when lost-phone mode is explicitly activated.
- Provide visible permission and foreground-service states for responsible operation.

## Implemented technology

The application is written in Kotlin with Jetpack Compose. CameraX supplies front-camera frames, MediaPipe Tasks Vision interprets a hand, the accessibility service performs cross-application actions, and Google Play Services Location supplies coordinates for the lost-phone workflow.

## 2. System Architecture

The project follows a four-layer path: capture input, interpret intent, orchestrate the selected feature, and execute an Android action. Permissions and user-enabled services form a trust boundary around the full path.



### Input layer

Microphone, front camera, accelerometer, proximity sensor, and the lost-mode secret-code field supply user intent.

### Interpretation layer

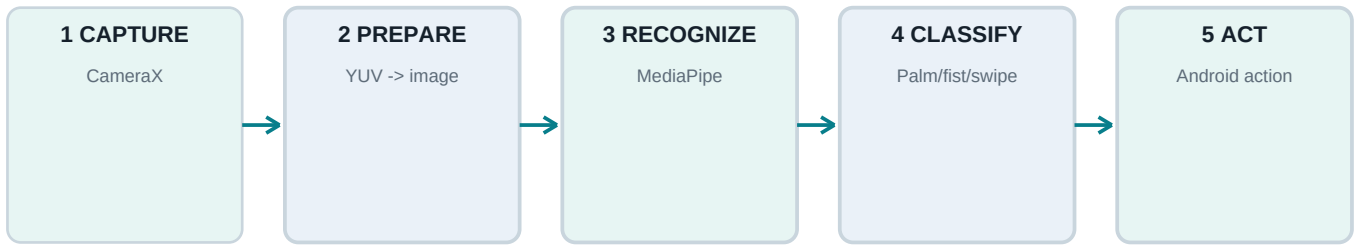
Android SpeechRecognizer provides candidate phrases. MediaPipe identifies built-in palm/fist categories and 21 normalized hand landmarks; landmark-center displacement identifies swipes. Sensor thresholds identify tilt, shake, and proximity transitions.

### Execution layer

AmmuAccessibilityService performs scroll, Back, Home, and Recents. AudioManager dispatches play/pause. TelecomManager handles supported call actions. The location service composes and sends email updates.

### 3. How Air Gestures Work

The front-camera workflow runs as a foreground service when the user enables Air Gestures. Processing retains only the latest camera frame to avoid a growing analysis queue.



| Swipe up    | Scroll down  | Accessibility forward scroll  |
|-------------|--------------|-------------------------------|
| Swipe down  | Scroll up    | Accessibility backward scroll |
| Swipe left  | Back         | Global Back action            |
| Swipe right | Recent apps  | Global Recents action         |
| Open palm   | Play / pause | Media key dispatch            |
| Closed fist | Home         | Global Home action            |

Scroll reliability: the service first searches the active accessibility tree for a scrollable node. When node scrolling fails, it dispatches a swipe gesture as a fallback.

## 4. Voice, Motion, and Call Control

### Voice flow

Tap To Talk starts SpeechRecognizer. The best recognized phrase is normalized and routed to a command handler. Commands cover YouTube, WhatsApp, Settings, accessibility navigation, media playback, calls, and feature controls. TextToSpeech and the Compose dashboard report the result.

| Stage     | Purpose                                        |
|-----------|------------------------------------------------|
| Listen    | Capture one command after explicit user action |
| Recognize | Receive candidate speech phrases               |
| Route     | Match a supported intent and arguments         |
| Execute   | Launch an intent or invoke an Android service  |
| Confirm   | Speak and display the status                   |

### Sensor controls

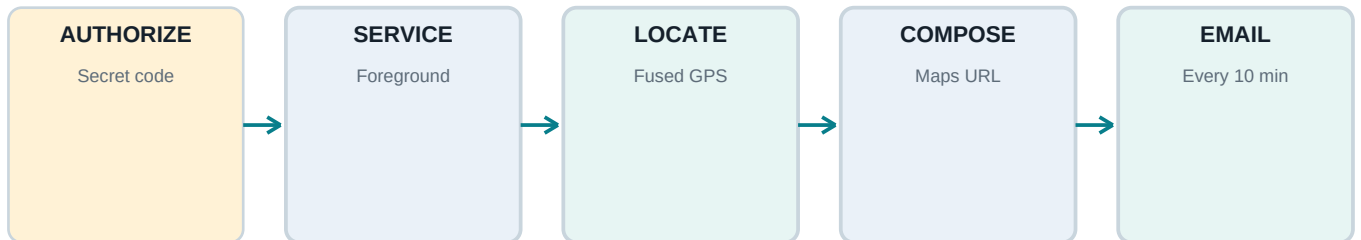
Accelerometer tilt maps to scrolling and navigation; shake maps to Recent Apps. The proximity sensor can support answer/end-call gestures when permissions, Android version, and manufacturer behavior permit it.

### Important limitation

Android's standard SpeechRecognizer is not a dedicated always-on wake-word engine. Continuous restart loops can cause microphone sounds and unreliable behavior, so the current documented operation is tap-to-talk.

## 5. Lost-Phone Location Workflow

Lost-phone tracking is an explicitly activated foreground feature. It checks the configured secret code, verifies location permission and SMTP settings, requests current coordinates, and sends a map link by email.



- The foreground notification remains visible and includes a Stop action.
- Google Play Services requests high-accuracy current location.
- The email contains latitude, longitude, accuracy, and a Google Maps URL.
- After an attempt, the service waits 10 minutes before the next update.
- Credentials and secret code are loaded from local.properties and excluded from Git.

### Security boundary

Use a Gmail App Password rather than a normal account password. Do not hard-code API keys, SMTP credentials, unlock PINs, or recipient details in source files. Rotate any credential previously shared publicly or in chat.

## 6. Project Requirements

| Development    | Android Studio; JDK 17+; Android SDK Platform 36; Build Tools; Platform Tools (adb); Git                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Android device | Android 7.0 / API 24 or newer; front camera; microphone; accelerometer/proximity where used; location services                      |
| Libraries      | Jetpack Compose; CameraX 1.5.3; MediaPipe Tasks Vision 0.10.20; Play Services Location 21.3.0; Coroutines 1.9.0; Android Mail 1.6.7 |
| Permissions    | Microphone; camera; fine/coarse location; notifications; phone state; answer calls; foreground camera/location; internet            |
| System setup   | USB debugging for installation; Ammu No Touch Control enabled in Accessibility settings                                             |

### Private configuration fields

sdk.dir, OPENAI\_API\_KEY, LOST\_PHONE\_EMAIL\_TO, LOST\_PHONE\_EMAIL\_FROM, LOST\_PHONE\_SMTP\_HOST, LOST\_PHONE\_SMTP\_PORT, LOST\_PHONE\_SMTP\_USERNAME, LOST\_PHONE\_SMTP\_PASSWORD, and LOST\_MODE\_SECRET\_CODE are read from local.properties. The example file documents the names without exposing values.

## 7. Build, Install, and Run

### Step 1 - Open and configure

Open the project folder in Android Studio. Copy `local.properties.example` to `local.properties`, verify `sdk.dir`, and add only the private values needed for the features you will test.

```
cd "/Users/munagapatibenarji/Documents/ammu projectair" ./gradlew :app:assembleDebug
```

### Step 2 - Connect the phone

Enable Developer Options and USB debugging. Connect the device by USB, approve the debugging prompt, and confirm that `adb` can see it.

```
adb devices
```

### Step 3 - Install

Run the app from Android Studio or install the debug build from Terminal.

```
./gradlew :app:installDebug
```

### Step 4 - Enable control

Grant requested runtime permissions. Open Android Accessibility settings and enable Ammu No Touch Control. Return to Ammu and start Air Gestures before testing hand control in another application.

### Step 5 - Functional test

Verify Tap To Talk, app launch, scroll up/down, Back, Home, Recents, media play/pause, then lost-phone tracking with a test email account. Keep the phone unlocked and visible during initial camera-gesture tuning.



## 8. Operating Guide and Limitations

### Example voice commands

Open YouTube; open WhatsApp; open Settings; scroll down; scroll up; go Home; open Recent Apps; play media; pause media; answer call; end call; start air gesture.

### Recommended gesture test

Use even lighting, keep one hand fully visible to the front camera, begin near the center of the view, and perform a deliberate single movement. Wait for the cooldown before repeating. Test Accessibility in a page that is actually scrollable.

### Current limitations

- Always-on wake-word behavior is not production-grade with SpeechRecognizer alone.
- Camera processing consumes battery and must remain transparent to the user.
- Accessibility behavior depends on the active app's UI hierarchy.
- Answer/end-call support varies by Android version and device manufacturer.
- SMTP credentials in an APK are recoverable; a secure backend is preferred for production.
- The project currently demonstrates rules-based gesture mapping rather than user-trained custom gestures.

### Suggested future work

Add device-specific calibration, automated command and gesture tests, a dedicated offline wake-word engine, lifecycle and thermal monitoring, secure server-mediated notifications, and an opt-in custom gesture training workflow.

### Project source

<https://github.com/munagapati-benarjibabu/Air-gesture-ammu>

This report reflects the repository implementation reviewed on 24 June 2026.